LAPORAN UJIAN TENGAH SEMESTER
PEMBANGUNAN PERANGKAT LUNAK BERORIENTASI SERVICE

SISTEM KEPEGAWAIAN & ABSENSI (Implementasi Arsitektur Microservices & Otentikasi Terpusat)

Disusun Oleh:

Nama:	Rafly Dzakki Pratama
NIM:	2310511066
Kelas:	A

PROGRAM STUDI INFORMATIKA

UNIVERSITAS PEMBANGUNAN NASIONAL VETERAN JAKARTA

2026

BAB I: PENDAHULUAN

1.1 Latar Belakang

Sistem Informasi Sumber Daya Manusia (HRIS) dan rekapitulasi absensi merupakan tulang punggung operasional di setiap institusi. Seiring dengan bertambahnya jumlah pegawai dan kompleksitas aturan jam kerja, sistem monolitik tradisional seringkali menghadapi masalah skalabilitas, terutama pada jam sibuk saat seluruh pegawai melakukan *clock-in* secara bersamaan. Untuk mengatasi *bottleneck* performa dan mengamankan data secara terisolasi, penerapan arsitektur *microservices* menjadi solusi yang tepat dan modern.

1.2 Tujuan

Tujuan dari pembangunan sistem ini adalah:

1. Merancang dan mengimplementasikan sistem kepegawaian berbasis *microservices* yang *scalable* dan modular.
2. Mengamankan *endpoint* API menggunakan otentikasi *JSON Web Token* (JWT) tersentralisasi dan integrasi *Single Sign-On* (SSO) melalui Google OAuth 2.0.
3. Memisahkan beban kerja pemrosesan data transaksional (kehadiran) dengan komputasi berat (pembuatan laporan agregat).

1.3 Tautan Demonstrasi

Dokumentasi visual dan panduan fungsionalitas sistem telah direkam dan dapat diakses melalui tautan berikut: Tautan Google Drive

BAB II: ARSITEKTUR SISTEM

Sistem ini dirancang menggunakan pendekatan terdistribusi, di mana setiap domain logika bisnis dipisah menjadi layanan (*service*) yang berdiri sendiri.

2.1 Komponen Microservices

1. API Gateway (Port 3000)
   Bertindak sebagai *Single Entry Point* atau pintu gerbang utama aplikasi. Gateway bertugas menerima semua *request* dari *client* (Postman/Frontend), melakukan *rate limiting* untuk mencegah serangan *spam*, memvalidasi keberadaan dan keabsahan JWT, lalu merutekan (*routing proxy*) permintaan ke *service* yang tepat.
2. Auth Service (Port 3001 - Node.js)
   Layanan khusus untuk menangani keamanan dan manajemen identitas. Menangani proses *login* lokal, penerbitan/pembaruan *Access* & *Refresh Token* (JWT), serta menjadi agen komunikasi pertukaran *authorization code* dengan *server* Google OAuth 2.0.
3. Employee & Attendance Service (Port 8000 - Laravel 11)
   Merupakan *core service* dari sistem ini. Dibangun dengan ekosistem PHP dan Laravel untuk memanfaatkan ketangguhan ORM (Eloquent) dalam mengelola relasi kompleks

antara data Pegawai, Jabatan, dan riwayat Absensi (*Clock-in*, *Clock-out* otomatis, telat, izin).

4. Report Service (Port 3002 - Node.js)
   Layanan pelaporan asinkron. Bertugas melakukan komputasi agregat (menghitung total hadir, telat, absen) dari keseluruhan data kehadiran setiap bulannya, dan menyimpannya ke dalam *database* mandiri tanpa mengganggu operasional harian di *Employee Service*.

BAB III: PANDUAN INSTALASI & PENGGUNAAN

3.1 Persyaratan Sistem (*Prerequisites*)

Pastikan lingkungan pengembangan (*development environment*) telah memenuhi spesifikasi berikut:

- PHP: Versi 8.3.30 atau lebih baru (beserta Composer).
- Node.js: Versi 24.13.1 atau lebih baru.
- Database: MySQL Server (dapat menggunakan XAMPP atau Laragon).
- API Client: Postman.

3.2 Persiapan Database

1. Jalankan layanan MySQL pada XAMPP/Laragon.
2. Buka antarmuka manajemen *database* (seperti phpMyAdmin atau melalui *command line*).
3. Buat tiga *database* kosong yang terisolasi untuk masing-masing layanan:
   - uts_auth_db
   - uts_employee_db
   - uts_report_db

3.3 Langkah Eksekusi Layanan (Dijalankan pada terminal terpisah)

Terminal 1: Menjalankan Employee Service (Laravel)

Arahkan direktori terminal ke dalam folder layanan pegawai, lalu eksekusi perintah inisialisasi:

```
cd services/employee-service

composer install

cp .env.example .env

php artisan key:generate

php artisan migrate --seed
```

php artisan serve

(Layanan akan aktif dan mendengarkan pada http://127.0.0.1:8000)

Terminal 2: Menjalankan Auth Service (Node.js)

Arahkan direktori terminal ke folder otentikasi:

cd services/auth-service

npm install

node index.js

(Layanan akan aktif dan mendengarkan pada http://localhost:3001)

Terminal 3: Menjalankan Report Service (Node.js)

Arahkan direktori terminal ke folder pelaporan:

cd services/report-service

npm install

node index.js

(Layanan akan aktif dan mendengarkan pada http://localhost:3002)

Terminal 4: Menjalankan API Gateway (Node.js)

Arahkan direktori terminal ke folder *gateway*:

cd gateway

npm install

node index.js

(Gateway akan aktif pada http://localhost:3000).

Catatan Penting: Seluruh pengujian *endpoint* melalui Postman wajib diarahkan ke Port 3000 agar dapat divalidasi oleh *Gateway* secara terpusat.

BAB IV: ANALISIS DAN JUSTIFIKASI ARSITEKTUR

Keputusan untuk menggunakan arsitektur *Microservices* dibandingkan *Monolithic* didasarkan pada analisis teknis terhadap tantangan aplikasi kepegawaian modern:

4.1 Isolasi Beban Kerja dan *Fault Tolerance* (Auth Service)

Pada arsitektur monolitik, lonjakan beban lalu lintas data (terutama pada jam operasional pagi hari ketika ratusan pegawai melakukan otentikasi secara bersamaan) dapat membebani CPU *server* utama. Dengan memisahkan **Auth Service**, proses kriptografi yang intensif (hashing *password*, verifikasi JWT, dan *handshake* dengan Google OAuth) ditangani oleh entitas komputasi yang terpisah. Keuntungan lainnya adalah *fault tolerance*; jika layanan otentikasi

mengalami gangguan sementara, komponen lain (seperti pelaporan asinkron) tetap dapat beroperasi di tingkat jaringan internal.

4.2 Efisiensi Kinerja Komputasi Berat (Report Service)

Pembuatan laporan rekapitulasi akhir bulan memerlukan agregasi data berskala besar (memetakan ratusan riwayat *clock-in* pegawai). Jika proses iterasi dan pemilahan *array* data ini dipaksakan berjalan pada *server* transaksional utama (Laravel), akan terjadi lonjakan konsumsi RAM yang dapat menyebabkan fitur *clock-in* harian menjadi lambat (*latency*) atau bahkan *timeout*. Dengan mendelegasikan tugas ini ke **Report Service**, perhitungan komputasi berat sepenuhnya diisolasi. *Report Service* menarik data menggunakan metode paginasi khusus (per_page=all) dan memprosesnya secara independen sebagai *background process*.

4.3 Kebebasan Pemilihan Teknologi (*Polyglot Persistence*)

Arsitektur terdistribusi memungkinkan perancangan sistem berbasis *Best-Tool-for-the-Job*:

- Laravel (Employee Service)
  Sangat ideal dan efisien untuk menangani struktur basis data relasional tingkat lanjut (Relasi *One-to-Many* pada tabel *Employees*, *Positions*, dan *Attendances*) berkat ekosistem ORM (Eloquent) dan *Form Request Validation* bawaannya.
- Node.js & Express (Gateway, Auth, Report)
  Diimplementasikan karena karakteristik *Event-driven* dan *Non-blocking I/O*. Sangat ringan dan secepat kilat untuk melakukan I/O jaringan (sebagai *Reverse Proxy* di Gateway), serta fleksibel dalam menangani manipulasi JSON mentah dan pustaka *third-party* seperti google-auth-library.

BAB V: KESIMPULAN

Pembangunan Sistem Kepegawaian & Absensi dengan arsitektur *microservices* telah berhasil mengimplementasikan lingkungan sistem yang skalabel, modular, dan tangguh terhadap disrupsi beban kerja. Penggabungan ekosistem Laravel untuk keandalan relasi data dan Node.js untuk asinkronisasi proses menghasilkan *software* dengan standar performa industri yang tinggi dan *security* yang kokoh (melalui enkripsi JWT dan Google OAuth 2.0 tersentralisasi pada API Gateway).